C++17

A Practical Guide for Students

Universidad Tecnológica Centroamericana — UNITEC

Ing. Iván de Jesús Deras Tábora

1. Introduction to C++17

C++ is a statically-typed, compiled, general-purpose language that gives the programmer direct control over memory and hardware. C++17, standardised in 2017, added many ergonomic features that make the language easier to use without sacrificing performance.

C++ is an excellent choice: it is fast, expressive, supports both low-level memory management and high-level abstractions, and its Standard Library provides all the data structures and algorithms you need.

1.2 Toolchain Setup

You need a C++17-capable compiler. Both GCC (≥ 7) and Clang (≥ 5) qualify.

```
# Compile with C++17
g++ -std=c++17 -Wall -Wextra -o myprogram main.cpp

# Or with Clang
clang++ -std=c++17 -Wall -Wextra -o myprogram main.cpp
```

> 💡 **Tip** Always compile with -Wall -Wextra during development. Compiler warnings catch real bugs.

1.3 A Minimal Program

```
#include <iostream>

int main() {
    std::cout << "Hello, Estructura de Datos II!" << std::endl;
    return 0;
}
```

2. Variables and Types

C++ is statically typed: every variable has a type known at compile time. The type determines how the value is stored in memory and which operations are valid on it.

2.1 Primitive Types

Type	Size / Description
`bool`	`true or false (1 byte)`
`char`	`Single character, 1 byte`
`int`	`Integer, typically 4 bytes`
`long`	`Integer, at least 4 bytes (often 8)`
`float`	`Single precision floating point, 4 bytes`
`double`	`Double precision floating point, 8 bytes`
`std::string`	`Text string (from <string>)`

```
bool   flag   = true;
char   letter = 'A';
int    count  = 42;
double pi     = 3.14159;
std::string name = "Compiladores";
```

2.2 const and constexpr

Use `const` for values that must not change after initialization. Use `constexpr` when the value is known at compile time — the compiler can use it in optimisations.

```
const int MAX_TOKENS = 1024;       // runtime constant
constexpr double PI  = 3.14159;    // compile-time constant
```

2.3 auto — Type Deduction

The `auto` keyword instructs the compiler to deduce the variable's type from its initialiser. This reduces verbosity without sacrificing type safety.

```
auto x = 42;          // int
auto y = 3.14;        // double
auto name = std::string("Lexer");  // std::string
auto it = myMap.begin();           // iterator type (complex, auto saves typing)
```

> ⚠ **Note** auto always requires an initialiser. auto x; alone is a compile error.

2.4 Type Aliases with using

The `using` keyword creates readable aliases for complex types.

```
using TokenId   = int;
using TokenList = std::vector<std::string>;

TokenId   id  = 5;
TokenList toks;
```

2.5 nullptr

`nullptr` is the type-safe null pointer constant introduced in C++11. Always prefer it over the old `NULL` macro or literal `0`.

```
int* p = nullptr;   // safe null pointer
if (p != nullptr) { /* ... */ }
```

2.6 References and Pointers

References are aliases for existing variables. Pointers store addresses. Both are crucial for efficient parameter passing.

```
int value = 10;
int& ref  = value;   // ref is an alias for value
int* ptr  = &value;  // ptr holds the address of value

ref = 20;            // value is now 20
*ptr = 30;           // value is now 30 (dereference)
```

> 💡 **Tip** Prefer references over pointers when the value is guaranteed to exist. Use pointers only when you need nullability or pointer arithmetic.

3. Control Flow

3.1 if / else

```
int x = 5;
if (x > 0) {
    std::cout << "positive";
} else if (x < 0) {
    std::cout << "negative";
} else {
    std::cout << "zero";
}
```

C++17 adds `if` with an initialiser, useful for narrowing scope:

```
// C++17: init-statement in if
if (auto it = myMap.find(key); it != myMap.end()) {
    std::cout << it->second;  // 'it' only lives inside this if
}
```

3.2 switch

```
char tok = '+';
switch (tok) {
    case '+': handleAdd(); break;
    case '-': handleSub(); break;
    case '*': handleMul(); break;
    default:  handleUnknown(); break;
}
```

3.3 Loops

while and do-while

```
int i = 0;
while (i < 10) { std::cout << i++; }

int j = 0;
do { std::cout << j++; } while (j < 10);
```

Traditional for loop

```
for (int i = 0; i < 10; ++i) {
    std::cout << i << ' ';
}
```

Range-based for (C++11+)

Iterate over any container or array cleanly:

```
std::vector<int> nums = {1, 2, 3, 4, 5};

for (int n : nums) {          // copy each element
    std::cout << n;
}

for (const auto& n : nums) {  // preferred: const reference, no copy
    std::cout << n;
}
```

> 💡 **Tip** Use const auto& in range-based for loops unless you need to modify the element or it is a cheap scalar type.

3.4 break, continue, return

- `break` – exits the innermost loop or switch immediately.
- `continue` – skips to the next iteration of the loop.
- `return` – exits the function, optionally returning a value.

4. Functions

4.1 Basic Syntax

```
// return_type name(parameters) { body }
int add(int a, int b) {
    return a + b;
}

void printLine(const std::string& msg) {
    std::cout << msg << '\n';
}
```

4.2 Parameter Passing

Style	Syntax & Behaviour
By value	`int f(int x)` — copy made, caller's variable unchanged
By reference	`int f(int& x)` — alias, caller's variable can be changed
By const ref	`int f(const int& x)` — alias, no copy, cannot modify
By pointer	`int f(int* x)` — pass address; can be null

For large objects (strings, vectors, custom classes) always pass by const reference to avoid expensive copies.

```
void process(const std::vector<int>& data) {  // no copy!
    for (const auto& v : data) { /* ... */ }
}
```

4.3 Default Arguments

```
void log(const std::string& msg, int level = 1) {
    std::cout << "[" << level << "] " << msg << '\n';
}

log("token found");       // level = 1
log("syntax error", 3); // level = 3
```

4.4 Function Overloading

Multiple functions may share a name if their parameter types differ. The compiler picks the best match.

```
double area(double radius);                    // circle
double area(double width, double height);      // rectangle
double area(double a, double b, double c);     // triangle (Heron)
```

4.5 inline Functions

`inline` suggests the compiler expand the function at the call site, eliminating call overhead. Use for tiny, frequently-called functions. Modern compilers inline aggressively regardless of the keyword.

```
inline int square(int x) { return x * x; }
```

4.6 Namespaces

Namespaces prevent name collisions in large projects by grouping related identifiers.

```
namespace lexer {
    int  nextToken();
    void reset();
}

// Using without prefix inside a scope:
using namespace lexer;
int t = nextToken();

// Or qualify explicitly (preferred in headers):
int t2 = lexer::nextToken();
```

> ⚠ **Note** Avoid using namespace std; in header files — it pollutes every translation unit that includes the header.

5. Classes and Object-Oriented Programming

A class bundles data (member variables) and behaviour (member functions) into a single unit. In C++ a class is identical to a struct except that members default to private.

5.1 Defining a Class

```
class Token {
public:
    // Constructor
    Token(std::string type, std::string value)
        : type_(std::move(type)), value_(std::move(value)) {}

    // Accessor (getter)
    const std::string& type()  const { return type_;  }
    const std::string& value() const { return value_; }

    // Mutator (setter)
    void setValue(std::string v) { value_ = std::move(v); }

private:
    std::string type_;
    std::string value_;
};

Token t("NUMBER", "42");
std::cout << t.type() << " " << t.value();
```

5.2 Constructors and Destructors

Constructors initialise an object when it is created. The destructor runs when the object goes out of scope — perfect for releasing resources.

```
class FileReader {
public:
    FileReader(const std::string& path) {
        file_.open(path);
        if (!file_) throw std::runtime_error("Cannot open " + path);
    }
    ~FileReader() { file_.close(); }  // called automatically

private:
    std::ifstream file_;
};
```

> 💡 **RAII** This pattern — acquiring a resource in the constructor and releasing it in the destructor — is called RAII (Resource Acquisition Is Initialization). It is the foundation of safe C++ programming.

5.3 Access Specifiers

- **public:** Accessible from anywhere. Use for the interface of the class.
- **private:** Accessible only within the class itself. Use for implementation details.
- **protected:** Accessible within the class and its subclasses. Rarely needed in practice.

In practice, almost all inheritance you will write uses public. Protected is occasionally useful for sharing internals with subclasses; private inheritance is a niche technique rarely seen outside library code.

5.4 static Members

`static` members belong to the class, not to any individual instance.

```
class IdCounter {
public:
    static int nextId() { return ++count_; }
private:
    static int count_;  // shared by all instances
};
int IdCounter::count_ = 0;  // definition outside class
```

6. Inheritance

Inheritance lets a derived class reuse and extend the behaviour of a base class. The most common form is public inheritance, which models an "is-a" relationship.

6.1 Public Inheritance (the Common Case)

```
class ASTNode {
public:
    virtual ~ASTNode() = default;
    virtual std::string toString() const = 0;  // pure virtual
};

class NumberNode : public ASTNode {
public:
    explicit NumberNode(double v) : value_(v) {}
```

```
    std::string toString() const override {
        return std::to_string(value_);
    }
private:
    double value_;
};

class BinaryOpNode : public ASTNode {
public:
    BinaryOpNode(char op, ASTNode* l, ASTNode* r)
        : op_(op), left_(l), right_(r) {}
    std::string toString() const override {
        return "(" + left_->toString() + op_ + right_->toString() + ")";
    }
private:
    char op_;
    ASTNode *left_, *right_;
};
```

6.2 virtual, override, and final

- `virtual` – marks a method as overridable in derived classes.
- `= 0` – makes the method pure virtual; the class becomes abstract.
- `override` – tells the compiler this method is intended to override a base-class virtual. A typo in the signature becomes a compile error instead of a silent new function.
- `final` – prevents further overriding (or class from being subclassed).

```
class Base {
public:
    virtual void draw() { /* default */ }
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void draw() override { /* specific */ }   // ✓
    // void drow() override { }               // ✗ compile error: no such base
method
};
```

> 💡 **Tip** Always declare destructors virtual in base classes that are used polymorphically. Without this, deleting a Derived* through a Base* causes undefined behaviour.

6.3 Constructor Chaining

Derived constructors must call the base constructor in the initialiser list:

```
class Animal {
public:
    Animal(std::string name) : name_(std::move(name)) {}
    virtual std::string speak() const = 0;
    const std::string& name() const { return name_; }
protected:
    std::string name_;
};

class Dog : public Animal {
public:
    Dog(std::string name) : Animal(std::move(name)) {}
    std::string speak() const override { return "Woof!"; }
};
```

7. Polymorphism

Polymorphism means "many shapes": the same function call produces different behaviour depending on the actual type of the object at runtime. In C++ this is achieved through virtual dispatch.

7.1 Runtime Polymorphism

```
void printNode(const ASTNode& node) {
    // Calls NumberNode::toString() or BinaryOpNode::toString()
    // depending on the actual object — decided at runtime
    std::cout << node.toString() << '\n';
}
```

The compiler builds a virtual dispatch table (vtable) for each class with virtual methods. Each object carries a hidden pointer to its class's vtable. The runtime looks up the correct function through this table — this is the mechanism behind polymorphism.

7.2 The Slicing Problem

Polymorphism only works through pointers or references, never through value copies. Copying a derived object into a base object discards the derived portion — this is called slicing.

```
Dog dog("Rex");

Animal a1 = dog;        // SLICING: a1 only has the Animal part
Animal& a2 = dog;       // OK: reference, no copy
Animal* a3 = &dog;      // OK: pointer, no copy

a2.speak();             // calls Dog::speak() correctly
```

⚠ **Warning** Store polymorphic objects through pointers or references. In modern C++ use smart pointers (see Chapter 11) instead of raw pointers.

7.3 Abstract Classes

A class with at least one pure virtual method cannot be instantiated directly. It serves as an interface contract.

```
class Visitor {                         // abstract — acts as interface
public:
    virtual void visitNumber(NumberNode&)   = 0;
    virtual void visitBinaryOp(BinaryOpNode&) = 0;
    virtual ~Visitor() = default;
};

// Visitor v;  ← compile error: cannot instantiate abstract class
```

8. Operator Overloading

C++ lets you define the meaning of built-in operators (==, +, <<, …) for your own types. Used well, operator overloading makes code read naturally. Used poorly, it makes code confusing.

8.1 Common Operators to Overload

Operator	Typical Use
`==  !=`	Equality comparison for use in containers and algorithms
`<  <=`	Ordering for std::map, std::sort
`<<`	Output to std::cout (stream insertion)
`[]`	Subscript / element access
`+ - * /`	Arithmetic on numeric wrapper types

8.2 Example: == and << on a Token

```
class Token {
public:
    Token(std::string type, std::string val)
        : type_(std::move(type)), val_(std::move(val)) {}

    bool operator==(const Token& o) const {
        return type_ == o.type_ && val_ == o.val_;
    }
    bool operator!=(const Token& o) const { return !(*this == o); }

    // Free function for stream insertion:
    friend std::ostream& operator<<(std::ostream& os, const Token& t) {
        return os << '[' << t.type_ << ':' << t.val_ << ']';
    }

private:
    std::string type_, val_;
};

Token a("NUM","1"), b("NUM","1");
std::cout << a << '\n';      // [NUM:1]
std::cout << (a == b);        // 1 (true)
```

💡 **Rule of Thumb** Only overload operators when their meaning is obvious and conventional. Defining + to mean "subtract" or << to mean "save to file" confuses everyone including your future self.

8.3 Comparison Operator < for Maps

If you want to use your type as a key in `std::map`, you must provide `operator<`:

```
struct Symbol {
    std::string name;
    int scope;
    bool operator<(const Symbol& o) const {
        if (name != o.name) return name < o.name;
```

```
        return scope < o.scope;
    }
};

std::map<Symbol, int> symbolTable;   // works because < is defined
```

9. std::string and std::string_view

9.1 std::string Essentials

```
#include <string>

std::string s = "Hello";
s += ", world";             // append
s.size();                   // 12
s.empty();                  // false
s.substr(0, 5);             // "Hello"
s.find(',');                // 5  (or std::string::npos if not found)
s[0];                       // 'H'
s.at(0);                    // 'H' with bounds checking
s.c_str();                  // const char* for C APIs
```

String concatenation with + creates temporary strings. For many concatenations prefer std::ostringstream or reserve() + append().

9.2 std::string_view (C++17)

`std::string_view` is a non-owning read-only view into a string (or any contiguous sequence of chars). It avoids copies and works with string literals, std::string, and raw buffers.

```
#include <string_view>

// Takes any string-like thing without copying:
void printUpper(std::string_view sv) {
    for (char c : sv) std::cout << (char)std::toupper(c);
}

printUpper("literal");          // no allocation
printUpper(std::string("obj")); // no copy
```

⚠ **Warning** Never return a string_view that refers to a local std::string — the string will be destroyed and the view will dangle.

10. Move Semantics and std::move

When an object is about to be destroyed or reassigned, C++ can "move" its internal resources to another object instead of copying them. This is a core performance feature of modern C++.

10.1 The Problem: Expensive Copies

```
std::vector<int> buildList() {
    std::vector<int> result;
    for (int i = 0; i < 1000000; ++i) result.push_back(i);
    return result;  // C++ moves (or elides) this automatically!
}

auto list = buildList();  // no million-element copy
```

10.2 std::move

`std::move` casts an object to an rvalue reference, allowing its resources to be taken (moved) rather than copied. After a move, the source is in a valid but unspecified state — don't use it again unless you reassign it.

```
#include <utility>

std::string a = "expensive string";
std::string b = std::move(a);   // b steals a's buffer; a is now empty

// Useful in constructors to avoid copying large members:
class Parser {
public:
    Parser(std::vector<Token> tokens)
        : tokens_(std::move(tokens)) {}  // move, not copy
private:
    std::vector<Token> tokens_;
};
```

10.3 When C++ Moves Automatically

- Returning a local variable from a function (NRVO / move).
- Inserting a temporary into a container.
- Passing a temporary to a `T&&` parameter.

> 💡 **Tip** In constructor initialiser lists, use std::move() when passing ownership of a parameter into a member. This is the single most common use of std::move in everyday code.

11. Smart Pointers

Raw pointers (int*) do not manage the lifetime of the object they point to. Smart pointers wrap raw pointers and automatically release memory when it is no longer needed. They are one of the most important tools for writing correct, leak-free C++.

11.1 Why Raw Pointers Are Dangerous

```
// Classic bugs with raw pointers:
ASTNode* node = new NumberNode(42);
// ... lots of code, an early return or exception occurs ...
delete node;   // might never be reached → MEMORY LEAK

ASTNode* p = new NumberNode(1);
delete p;
p->toString(); // USE AFTER FREE — undefined behaviour

ASTNode* q = new NumberNode(2);
delete q;
delete q;      // DOUBLE FREE — undefined behaviour
```

11.2 unique_ptr — Sole Ownership

`std::unique_ptr<T>` owns its object exclusively. When the unique_ptr is destroyed (goes out of scope), the object is automatically deleted. Ownership can be transferred but not shared.

```
#include <memory>

// Creation — prefer make_unique
auto node = std::make_unique<NumberNode>(42.0);

// node owns the object; no manual delete needed.
// When node goes out of scope, ~NumberNode() is called automatically.

// Transfer ownership:
auto owner2 = std::move(node);  // node is now null

// Returning from a factory function:
std::unique_ptr<ASTNode> makeNumber(double v) {
    return std::make_unique<NumberNode>(v);
}
```

💡 **Rule** Use unique_ptr as the default smart pointer. It has zero overhead compared to a raw pointer and eliminates the entire class of ownership bugs.

11.3 shared_ptr — Shared Ownership

`std::shared_ptr<T>` uses reference counting: it tracks how many shared_ptrs point to the same object. The object is deleted when the last shared_ptr pointing to it is destroyed.

```
auto s1 = std::make_shared<NumberNode>(3.14);
{
    auto s2 = s1;  // ref count = 2
    std::cout << s2->toString();
}  // s2 destroyed, ref count = 1 — object still alive
// s1 destroyed here, ref count = 0 — object deleted
```

shared_ptr is appropriate when multiple owners genuinely share an object — for example, a symbol table entry referenced from multiple AST nodes.

11.4 weak_ptr — Non-Owning Observation

`std::weak_ptr<T>` observes a shared_ptr-managed object without participating in ownership. It does not keep the object alive. You must lock() it to get a shared_ptr before using the object — this safely handles the case where the object has already been deleted.

```
auto s = std::make_shared<NumberNode>(1.0);
std::weak_ptr<NumberNode> w = s;

// Later, safely check if still alive:
if (auto locked = w.lock()) {
    std::cout << locked->toString();
} else {
    std::cout << "Object is gone";
}
```

💡 **Use weak_ptr to break cycles** If two objects hold shared_ptrs to each other, neither is ever deleted (circular reference). Make one side a weak_ptr to break the cycle.

11.5 Smart Pointer Quick Reference

Pointer	Ownership / Use
`unique_ptr<T>`	Single owner. Zero overhead. Default choice.
`shared_ptr<T>`	Multiple owners. Reference counted. Use when truly shared.
`weak_ptr<T>`	No ownership. Breaks reference cycles or caches observation.

12. Lambda Expressions

A lambda is an anonymous function defined inline at the point of use. Lambdas are indispensable for customising STL algorithms and for concise callbacks.

12.1 Syntax

```
// [capture](parameters) -> return_type { body }

auto greet = []() { std::cout << "Hello!"; };
greet();  // Hello!

auto add = [](int a, int b) { return a + b; };  // return type deduced
std::cout << add(3, 4);  // 7
```

12.2 Captures

Captures give the lambda access to variables from the surrounding scope.

Capture	Meaning
`[]`	Capture nothing
`[x]`	Capture x by value (copy)

Capture	Meaning
`[&x]`	Capture x by reference
`[=]`	Capture all used variables by value
`[&]`	Capture all used variables by reference
`[=, &x]`	Capture all by value, but x by reference

```
int threshold = 5;
auto bigEnough = [threshold](int v) { return v > threshold; };

std::vector<int> nums = {1, 4, 6, 2, 9};
auto it = std::find_if(nums.begin(), nums.end(), bigEnough);
// *it == 6
```

12.3 Lambdas with STL Algorithms

```
#include <algorithm>

std::vector<std::string> tokens = {"if","while","return","int"};

// Sort alphabetically:
std::sort(tokens.begin(), tokens.end());

// Sort by length:
std::sort(tokens.begin(), tokens.end(),
    [](const std::string& a, const std::string& b){
        return a.size() < b.size();
    });

// Count tokens with length > 3:
int n = std::count_if(tokens.begin(), tokens.end(),
    [](const std::string& s){ return s.size() > 3; });
```

13. STL Containers

The Standard Template Library provides ready-made, generic containers. Choose the right container for the job and you get correct, efficient code with minimal effort.

13.1 std::vector — Dynamic Array

The most frequently used container. Stores elements contiguously in memory (like an array) but grows automatically.

```
#include <vector>

std::vector<int> v = {10, 20, 30};
v.push_back(40);          // append
v.pop_back();             // remove last
v.size();                 // 3
v[1];                     // 20 (no bounds check)
v.at(1);                  // 20 (bounds checked)
v.front();  v.back();     // first / last element
v.clear();                // remove all elements
```

```
v.reserve(100);           // pre-allocate for 100 elements

// Iterate:
for (const auto& x : v) std::cout << x << ' ';
```

13.2 std::list — Doubly-Linked List

Efficient O(1) insertion and removal anywhere in the list, but no random access. Use when you frequently insert or remove elements in the middle.

```
#include <list>

std::list<std::string> l = {"a", "b", "c"};
l.push_front("z");        // insert at front
l.push_back("d");         // insert at back
l.insert(l.begin(), "X"); // insert before iterator
l.remove("b");            // remove all elements equal to "b"
```

13.3 std::map — Sorted Key/Value

Stores key-value pairs sorted by key. Lookups, insertions, and deletions are O(log n). Key type must support operator<.

```
#include <map>

std::map<std::string, int> symbolTable;
symbolTable["x"]  = 1;
symbolTable["y"]  = 2;

// Safe lookup:
auto it = symbolTable.find("x");
if (it != symbolTable.end()) {
    std::cout << it->second;  // 1
}

// Iterate in sorted key order:
for (const auto& [key, val] : symbolTable)  // structured binding
    std::cout << key << '=' << val << '\n';
```

13.4 std::unordered_map — Hash Map

Same interface as map, but uses hashing. O(1) average lookup/insert. Keys must be hashable; order is not guaranteed.

```
#include <unordered_map>

std::unordered_map<std::string, int> freq;
freq["if"]++;
freq["while"]++;
std::cout << freq["if"];  // 1
```

💡 **Choose wisely** Use map when you need sorted order or range iteration. Use unordered_map when you only need fast key lookup and order does not matter.

13.5 std::set and std::unordered_set

Sets store unique keys only — no values. std::set is sorted; std::unordered_set uses hashing.

```
#include <set>
#include <unordered_set>
```

```
std::set<std::string> keywords = {"if","while","return"};
keywords.insert("for");
keywords.count("if");      // 1 (present)
keywords.count("goto");    // 0 (absent)

std::unordered_set<int> seen;
seen.insert(42);
bool already = seen.count(42) > 0;  // true
```

13.6 std::queue and std::stack

```
#include <queue>
#include <stack>

// FIFO queue
std::queue<Token> tokenQueue;
tokenQueue.push(t1);
tokenQueue.front();  // peek
tokenQueue.pop();    // remove front

// LIFO stack
std::stack<int> stateStack;
stateStack.push(0);
stateStack.top();    // peek
stateStack.pop();    // remove top
```

14. std::optional and std::variant

14.1 std::optional — Nullable Values

`std::optional<T>` holds either a value of type T or nothing (`std::nullopt`). It makes the absence of a value explicit in the type system, replacing null pointers and sentinel values like -1.

```
#include <optional>

std::optional<int> findSymbol(const std::string& name) {
    if (table.count(name)) return table.at(name);
    return std::nullopt;               // nothing found
}

auto result = findSymbol("x");

// Check and use:
if (result.has_value()) {
    std::cout << *result;              // dereference like a pointer
}

// Shorthand:
if (result) { std::cout << *result; }

// Default if absent:
int val = result.value_or(0);
```

💡 **Tip** std::optional is ideal for function return values where failure is a normal, expected outcome — for example, symbol table lookups, parser peek-ahead, and config queries.

14.2 std::variant — Type-Safe Union

`std::variant<T1, T2, ...>` holds exactly one value, which can be any of the listed types. It is a type-safe replacement for C unions and for base-class pointer hierarchies where the set of types is fixed and known.

```
#include <variant>

using Value = std::variant<int, double, std::string>;

Value v1 = 42;
Value v2 = 3.14;
Value v3 = std::string("hello");
```

Accessing the Value

```
// std::get — throws if wrong type:
int n = std::get<int>(v1);

// std::get_if — returns pointer, null if wrong type:
if (auto* p = std::get_if<int>(&v1)) {
    std::cout << *p;
}

// std::visit — dispatches to correct overload automatically:
std::visit([](auto&& arg) {
    std::cout << arg;
}, v1);
```

Pattern Matching with std::visit

```
struct Printer {
    void operator()(int i)                { std::cout << "int: "    << i; }
    void operator()(double d)             { std::cout << "double: " << d; }
    void operator()(const std::string& s){ std::cout << "string: " << s; }
};

std::visit(Printer{}, v3);  // prints: string: hello
```

💡 **Compiler relevance** variant is perfect for representing a Token's semantic value: it can be an int, double, or string depending on the literal, all in one type-safe package.

15. Structured Bindings

C++17 structured bindings let you unpack a `std::pair`, `std::tuple`, array, or any struct with public members into named variables in a single declaration.

15.1 With Pairs and Tuples

```
#include <tuple>

std::pair<std::string, int> getToken() {
    return {"NUMBER", 42};
}

auto [type, value] = getToken();
std::cout << type << ' ' << value;  // NUMBER 42

// Tuple:
std::tuple<int, double, std::string> t = {1, 3.14, "hello"};
auto [a, b, c] = t;
```

15.2 Iterating Maps

This is the most common real-world use — iterating key/value maps becomes much more readable:

```
std::map<std::string, int> symbolTable = {{"x",1},{"y",2}};

// Without structured bindings:
for (const auto& entry : symbolTable)
    std::cout << entry.first << '=' << entry.second;

// With structured bindings (C++17):
for (const auto& [name, address] : symbolTable)
    std::cout << name << '=' << address;
```

15.3 With Structs

```
struct Token { std::string type; int line; int col; };

Token tok{"IF", 10, 3};
auto& [type, line, col] = tok;   // binds by reference
std::cout << type << " at " << line << ':' << col;
```

16. Error Handling

16.1 Exceptions

```
#include <stdexcept>

void openFile(const std::string& path) {
    std::ifstream f(path);
    if (!f) throw std::runtime_error("Cannot open: " + path);
}

try {
    openFile("tokens.txt");
} catch (const std::runtime_error& e) {
    std::cerr << "Error: " << e.what() << '\n';
} catch (...) {   // catch-all
    std::cerr << "Unknown error\n";
}
```

16.2 noexcept

`noexcept` declares that a function will not throw. It enables optimisations and documents intent. Destructors are implicitly noexcept.

```
int safeAdd(int a, int b) noexcept { return a + b; }

// Move constructors should always be noexcept:
class Buffer {
public:
    Buffer(Buffer&&) noexcept;  // allows STL to move instead of copy
};
```

16.3 Errors vs Exceptions: When to Use Each

- **Exceptions** – for truly exceptional conditions: file not found, network failure, invariant violation. They propagate automatically up the call stack.
- **Return values** – use `std::optional` or `bool` + output parameter for expected failures: token not in table, pattern not matched.
- **assert()** – for internal invariants that should never fail. Disabled in release builds.

Appendix: Quick Reference

A.1 Container Comparison

Container	Access	Insert	Remove	Ordered?
`vector<T>`	O(1) index	O(1)* back	O(n) mid	No
`list<T>`	O(n)	O(1) any	O(1) any	No
`map<K,V>`	O(log n)	O(log n)	O(log n)	Yes (key)
`unordered_map<K,V>`	O(1) avg	O(1) avg	O(1) avg	No
`set<K>`	O(log n)	O(log n)	O(log n)	Yes
`unordered_set<K>`	O(1) avg	O(1) avg	O(1) avg	No
`queue<T>`	front/back	O(1) back	O(1) front	No
`stack<T>`	top only	O(1) top	O(1) top	No

A.2 Smart Pointer Cheat Sheet

Smart Pointer	When to use
`make_unique<T>()`	Default. Single owner. Zero overhead.
`make_shared<T>()`	Genuinely shared ownership. Ref counted.
`weak_ptr<T>`	Observe without owning. Break cycles.

A.3 C++17 Feature Index

- `auto` – type deduction for variables (C++11, but essential)
- `if (init; cond)` – initialiser inside if/switch (C++17)
- `string_view` – non-owning string reference (C++17)
- `optional<T>` – nullable value (C++17)
- `variant<T...>` – type-safe union (C++17)
- `auto [a,b] = pair` – structured bindings (C++17)
- `make_unique<T>()` – preferred smart pointer factory (C++14+)